

Agenda

- Performance Metrics
- Reliability
- Caching



UTEC

PERFORMANCE

ARQUITECTURA DE SOFTWARE

Carlos Balbuena

Que es Performance ?

It depends

Define cual es tu
load ?

LOAD = USERS



SISTEMA

USERS siempre es la carga(load) correcta ?

#USERS = 10

LOAD = USERS



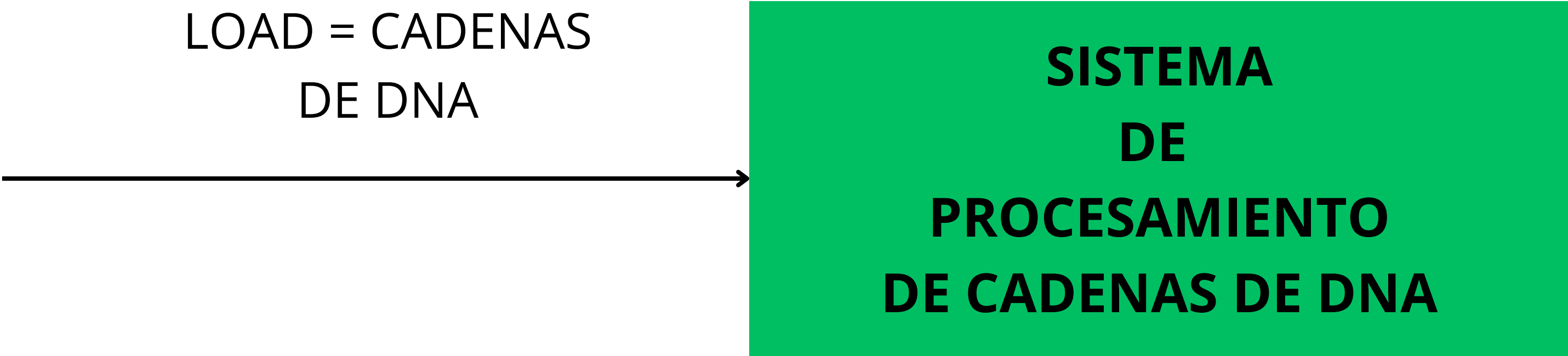
**SISTEMA
DE
PROCESAMIENTO
DE CADENAS DE DNA**

USERS siempre es la carga(load) correcta ?

#USERS = 10

#CADENAS DE DNA = 1 MILLON / Hora

LOAD = CADENAS
DE DNA



```
graph LR; A[LOAD = CADENAS DE DNA] --> B[SISTEMA DE PROCESAMIENTO DE CADENAS DE DNA];
```

**SISTEMA
DE
PROCESAMIENTO
DE CADENAS DE DNA**

USERS siempre es la carga(load) correcta ?

Entonces para un sistema de procesamiento de DNA

Si tu LOAD esta basada en USERS. Decir que tiene buena performance es incorrecto.

Esto a causa de que el sistema es estresado por las cadenas de DNA

Pregunta

En el sistema de Leasing
que Load elegiria
para medir performance ?

Pregunta

En el sistema de colegios
Cual seria el load que elegiria ?

Ok tengo mi load seleccionada
Whats next ?



**Que pasa
en mi arquitectura
si incremento:**

Usuarios

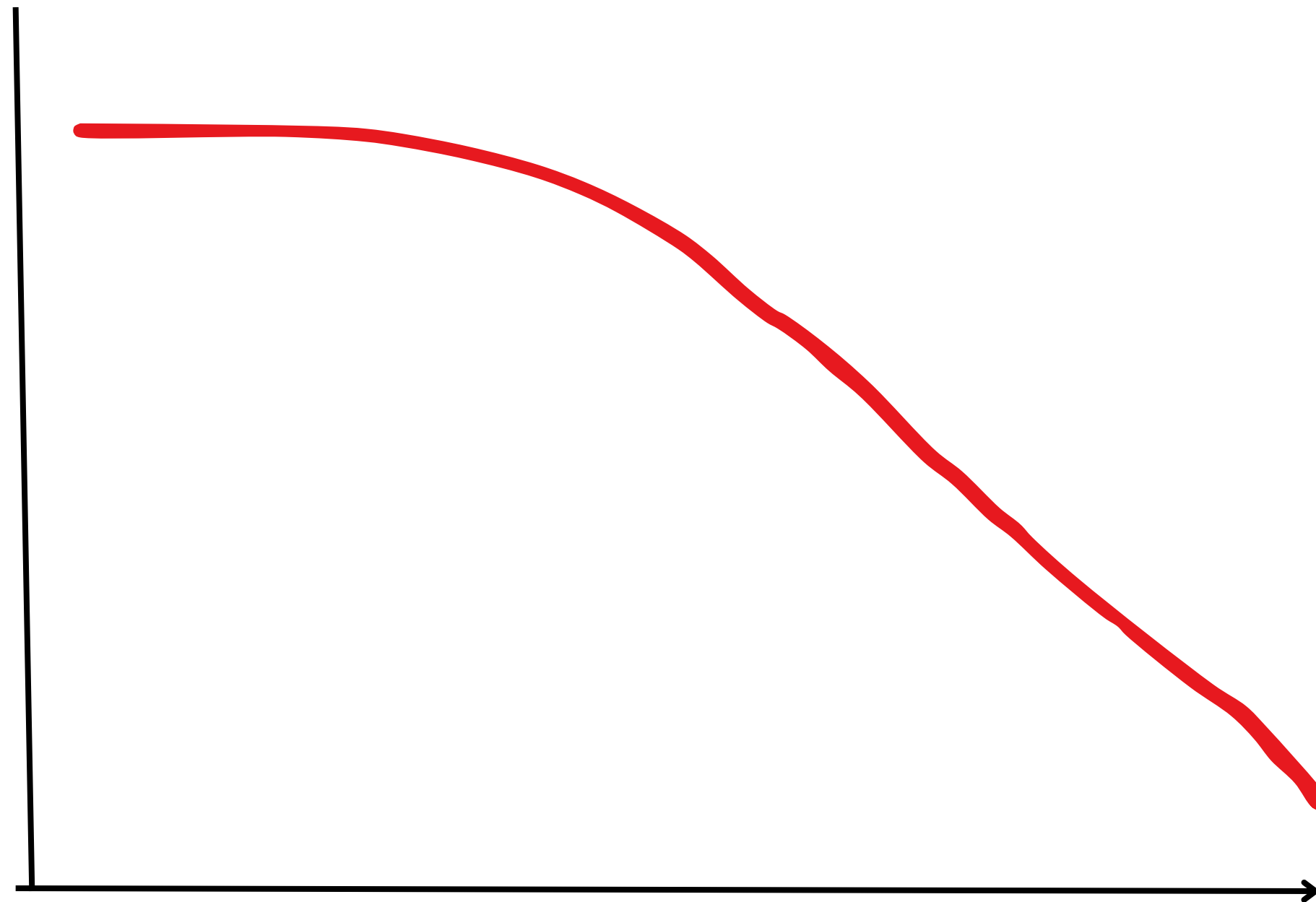
Documentos

Videos

Bytes ?

Que no debe pasar =>

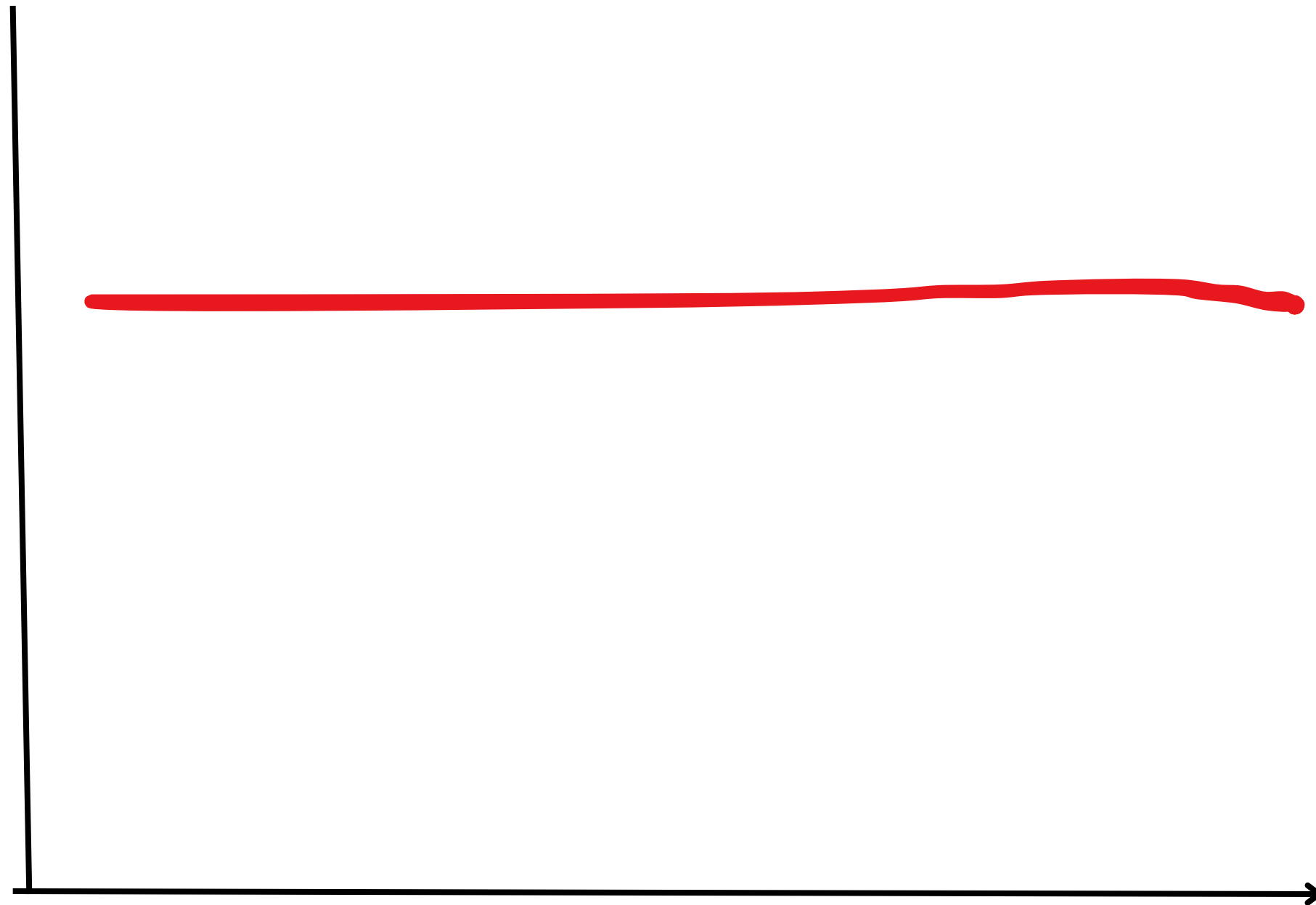
Load
(e.g. usuarios)



Performance

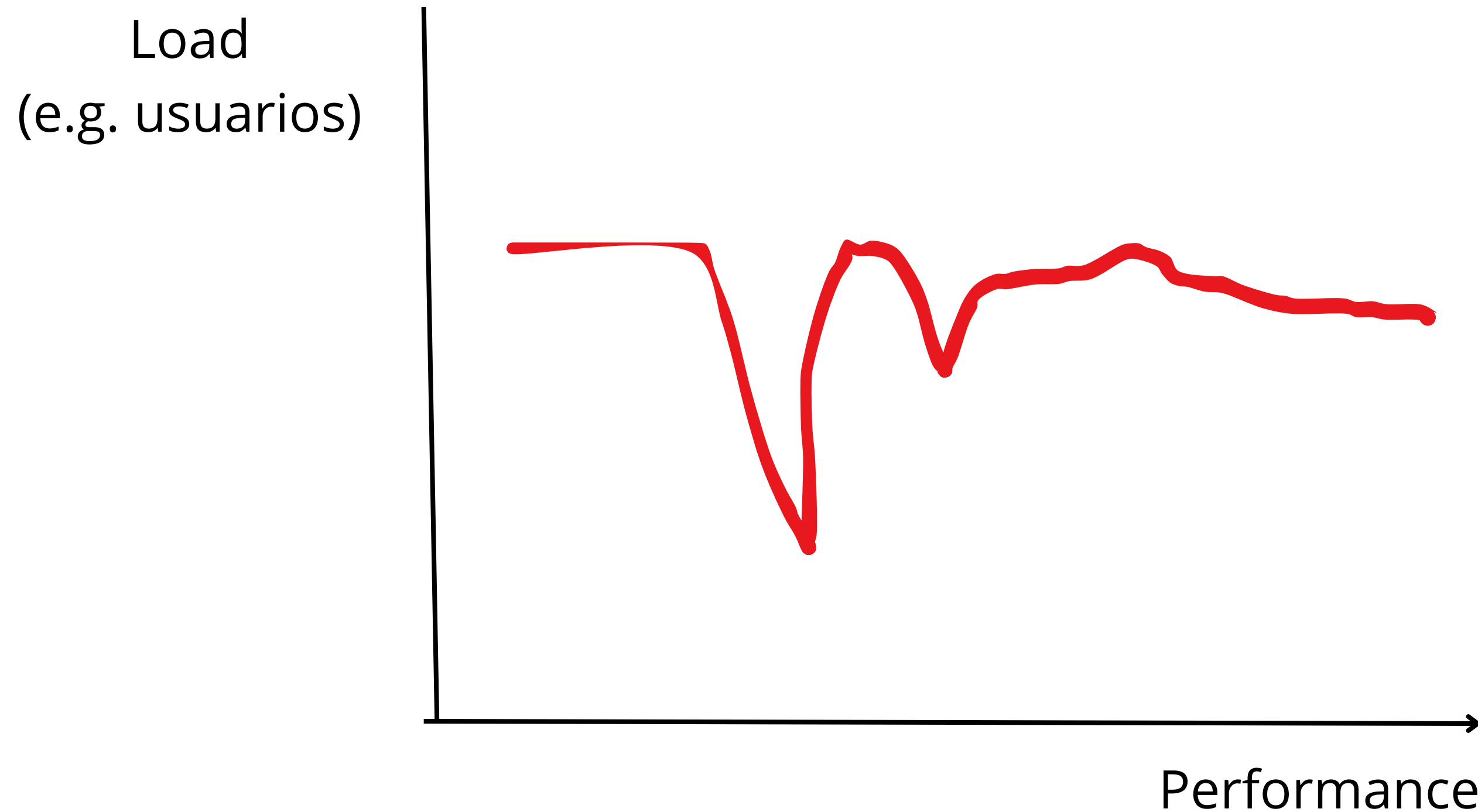
Que deberia pasar =>

Load
(e.g. usuarios)



Performance

Que pasa en realidad =>



Recorder

Throughput = number of X / seconds

X: Users,
Documents,
Records

Recordar

Response Time = Tiempo que demora un request en el sistema

Latency = Tiempo un request esta en espera a ser atendido o latente

(*) En general Latency se usa como si fuera igual a response time pero son diferentes.

Como mido mi performance ?

Request 1: 100ms

Request 2: 400 ms

Request 3: 1 ms

Request 4: 0ms

Request 5: 500ms

Como mido mi performance ?

El usuario 1 le toca una performance de 0 ms y es rapido ?

El usuario 5 le toca una performance de 400 ms y es lento ?

Como mido mi performance ?

No se puede medir por impresiones
individuales

**Tenemos que analizar una distribución
de la data a lo largo del tiempo**

Percentiles !

P95 = 1.5s

95 de 100 Requests $\leq$ 1.5s

5 de 100 $>$ 1.5 s

P99 = 1.5s

99 de 100 Requests $\leq$ 1.5s

1 de 100 $>$ 1.5 s



Y por que me importa que solo 1
de 100 tenga alta latencia ?

First

$$1\% \text{ de } 100 = 1$$

...

$$1\% \text{ de } 1000000 = 10000$$

etc..

Second

Ese 1 o 5% que demora o consume mas tiempo significa que su use case esta consumiendo mas. Un ejemplo simple su shopping cart es de 40 items.

Entonces este cliente es valioso \$_\$

Es decir:

Los P95 y P99
son el punto base de la
experiencia que le
daremos a nuestros
usuarios/clientes

Es decir:

O sea si tu P95 es 400 ms

Significa que 95% esta
recibiendo 100, 200, 300 ms?

**Lo cual no es un buen
indicador**

Medidas estandar de Performance (backend API edition)

$$\text{Availability} = 2xx / (2xx + 5xx)$$

(server side only)

$$\text{Reliability} = 2xx / (2xx + 4xx + 5xx)$$

(client and server side)

Preguntas : Performance (5 min)

- Determine cual es el load de su sistema de colegio.
- Determine cual es el load de su sistema de Leasing.



UTEC

RELIABILITY

ARQUITECTURA DE SOFTWARE

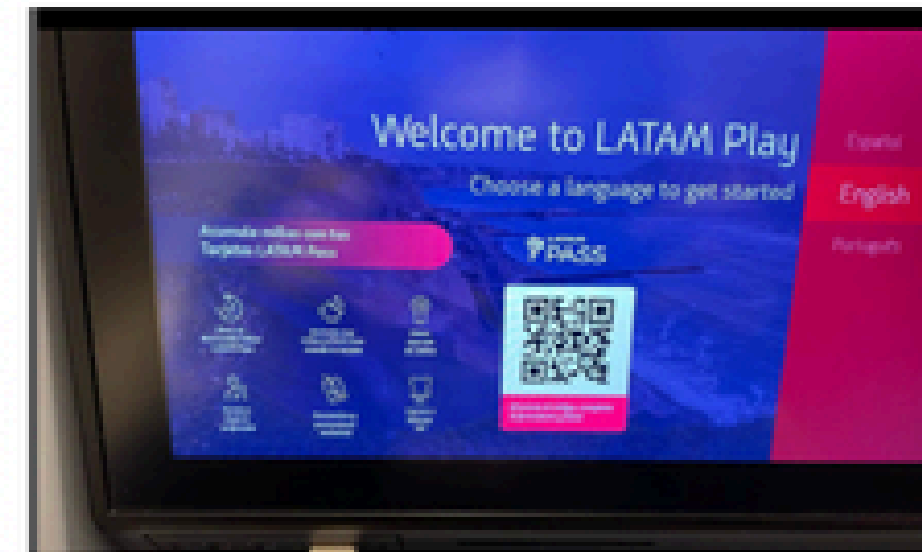
Carlos Balbuena

Entendiendo Le Reliability !

Traducción al español: **Confiabilidad**



Entendiendo Reliability



¿ Si ustedes desarrollan el software de un avión ? ¿ Subirían a ese avión ?

Famous Software Disasters !!

Medical Machine Kills (1985)

Cost: Three people dead, three people critically injured

Disaster: Canada's Therac-25 radiation therapy machine malfunctioned and delivered lethal radiation doses to patients.

Cause: Because of a subtle bug called a [race condition](#), a technician could accidentally configure Therac-25 so the electron beam would fire in high-power mode without the proper patient shielding.



Source: <https://www.devtopics.com/20-famous-software-disasters/>

Y2k - End of The World - Finales de 1999





Y2K - Cuál era el error ?

Anteriormente, el espacio de almacenamiento era un recurso que no estaba disponible en cantidades abundantes. En ese sentido, **ahorrar bits** era una práctica cotidiana y necesaria para no incrementar aún más los costos del proyecto. Es así como los programadores de la época decidieron reducir el campo del año en la fecha a dos dígitos. **Este ahorro de espacio parecía no considerar que los programas seguirían siendo usados 20 años después.**

En resumen: el campo año solo tenía la capacidad de guardar hasta 99(1999) ? Llegó el 2000 y ahora ? Todos los bancos van a caer ! All systems are doomed !



Y2k ?

Good news: No se acabó el mundo

Bad news: corregir ese bug costó un total de **300.000 millones** de dólares a nivel mundial.

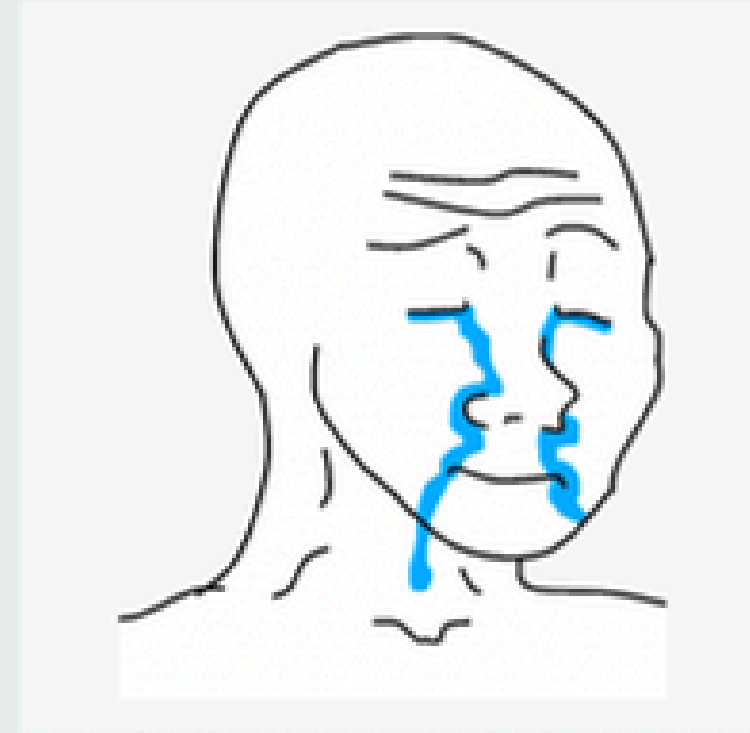


Entendiendo Reliability

¿ Por qué en software está permitido tener un reliability **tan bajo** ?
Es decir porque tenemos software con tantas fallas ????

Entonces qué hacemos ?

Aceptar la realidad:



**Casi el 100% del software va
fallar**

Le fiabilité à la rescousse => Reliability to the rescue !!

Reliability:

<< Continuing to work **correctly**, even when things **go wrong** >>

Traducción: Fallar con Elegancia





Reliability - Know your vocabulary

Things that go wrong => **Faults**

- *what is wrong ? Wrong is software having unexpected behaviours.*

Fault-Tolerant System => **Reliable system**



Fault Types

- Hardware Faults
 - Hard Disk, Memory, Network, Corrupted Database.
- Software Errors
 - OutOfMemory, NullPointerExceptions, 500 responses from external APIs, Concurrency.
- Human Errors
 - Edge cases, Unexpected scenarios.

Source: Designing Data-Intensive Applications - Martin Kleppmann



¿Cuán importante es tener Fault-Tolerant Systems ?

Super:

- Un software con comportamiento aleatorio puede causar pérdidas de dinero o tiempo cuando se usa.

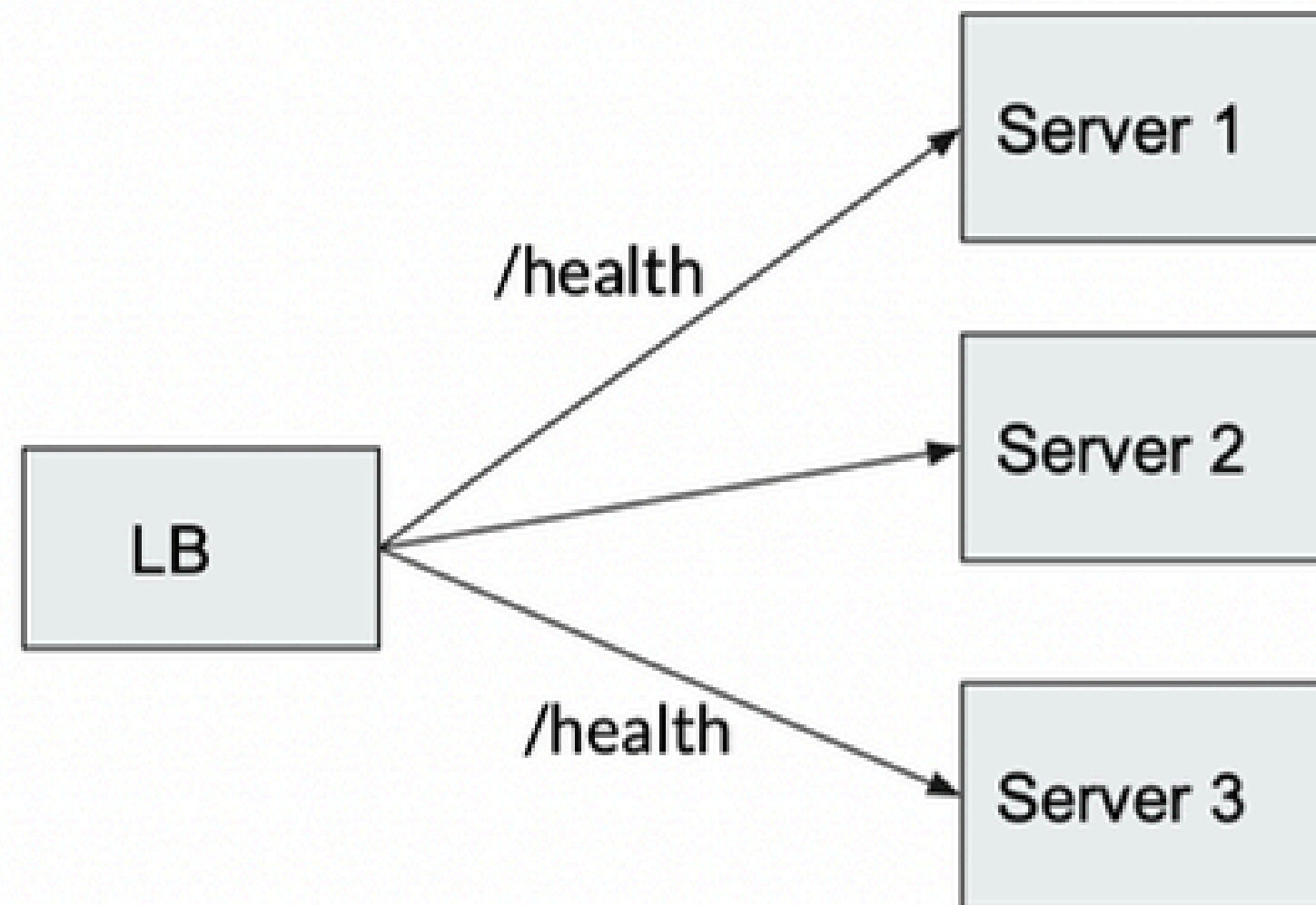
Es nuestra responsabilidad con nuestros usuarios entregarles un software que funcione al menos un 99.9% de las veces.

Técnicas de Reliability

Health Endpoints

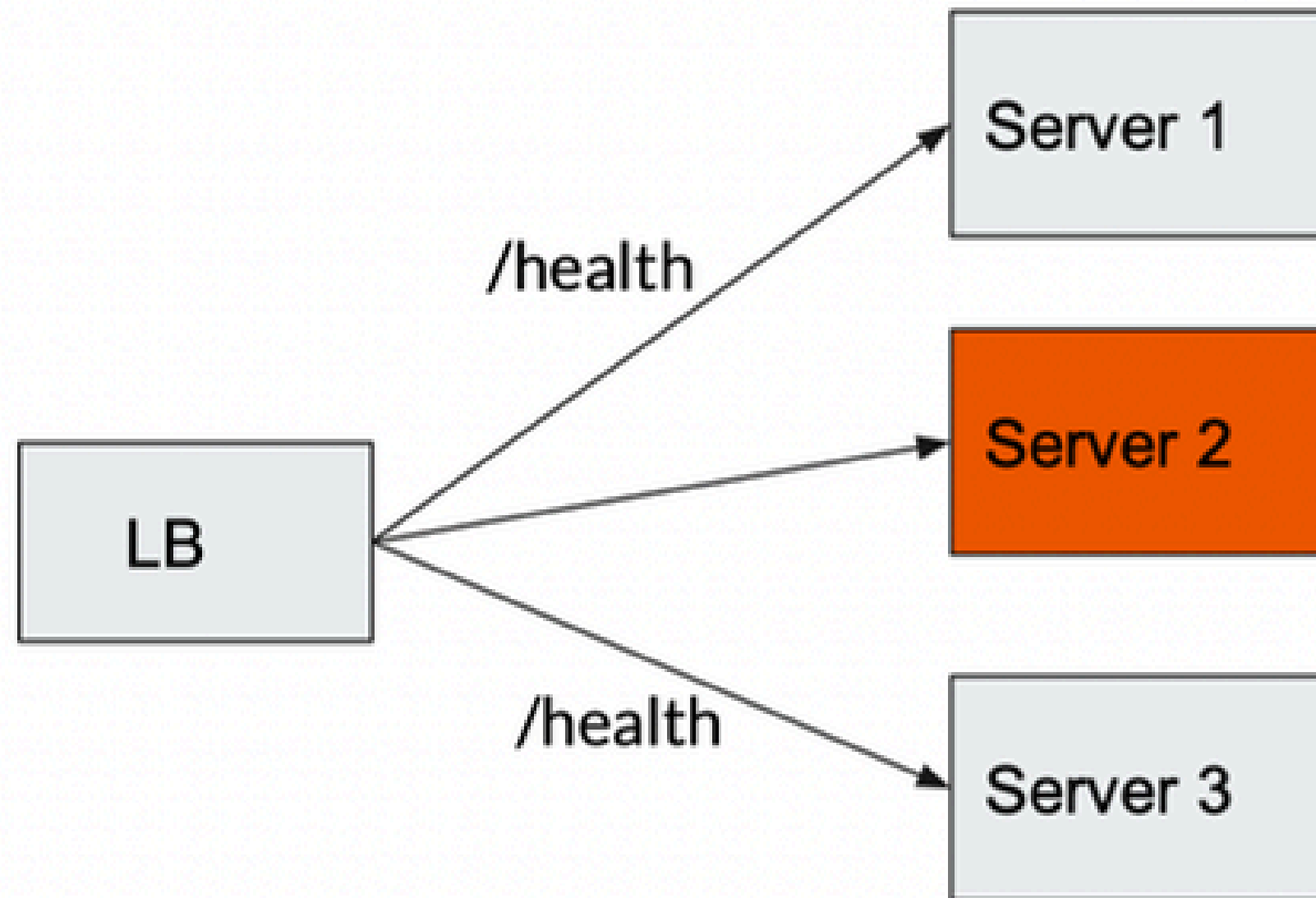
Load Balancing

Server1	Healthy
Server2	Healthy
Server3	Healthy



Load Balancing

Server1	Healthy
Server2	Unhealthy
Server3	Healthy

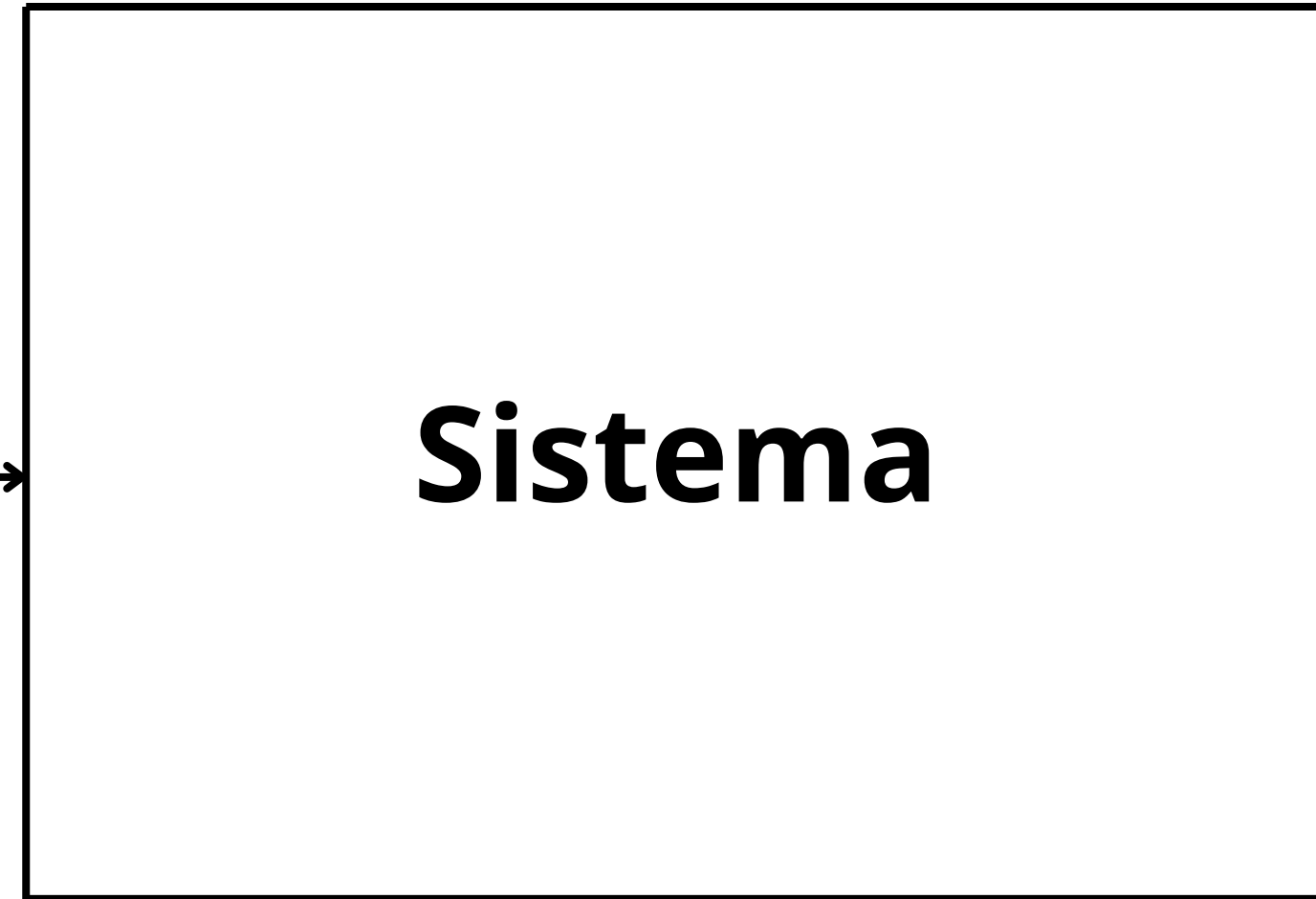


**Circuit
Breaker <3**

10 - Requests 200 OK



1 - Requests 500 FAIL

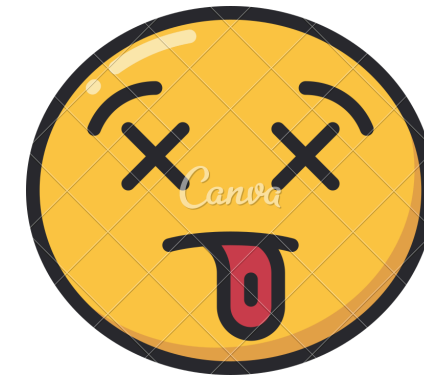


En base a nuestro analisis de performance
encontramos que el sistema se vuelve lento a partir de los
70 requests fallidos (500)

1 - Requests 200 OK

70 - Requests 500 FAIL

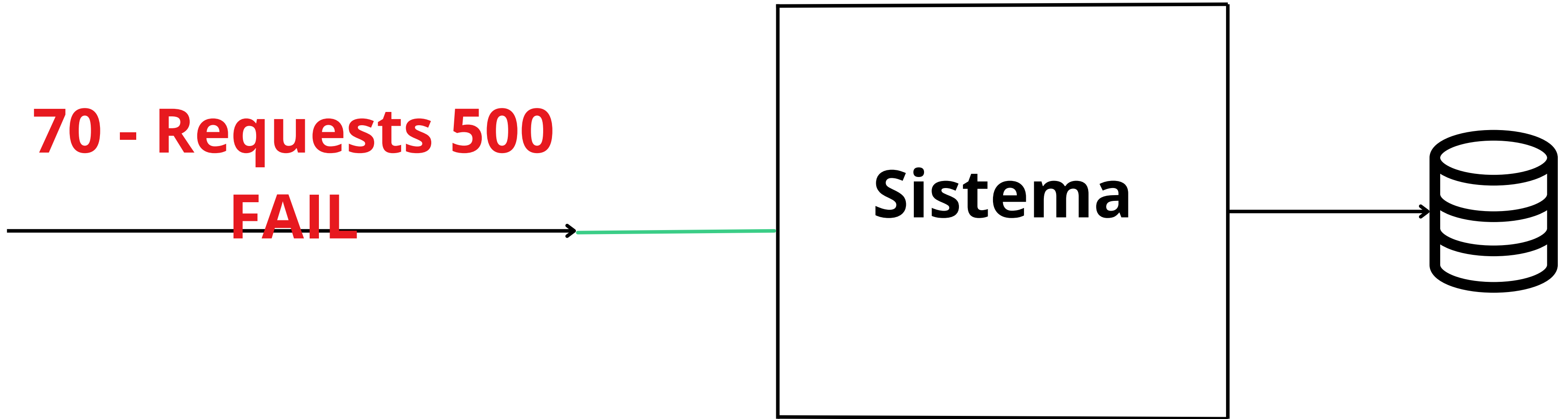
Sistema



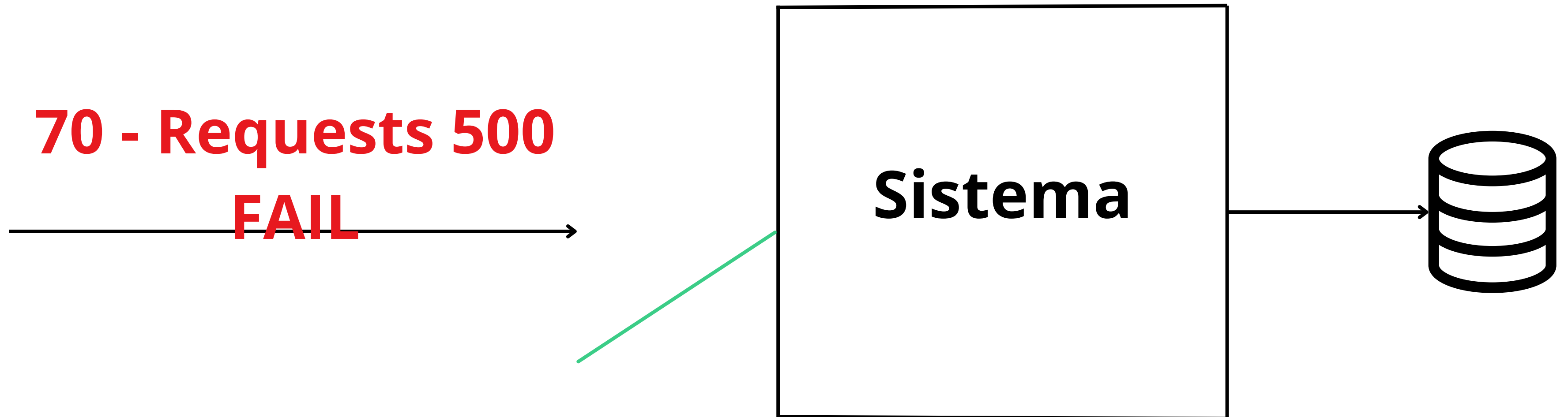
Oh no ! Que hacemos Sr Stark ?

70 - Requests 500

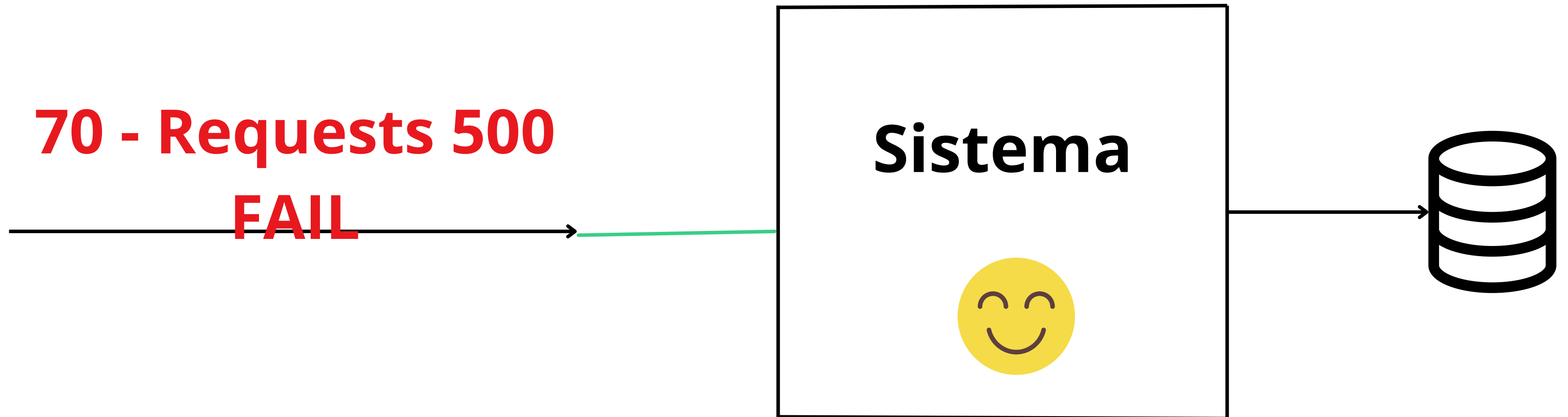
FAIL



Vamos asumir que el sistema se pone lento porque hay muchos locks y la BD se bloquea



Que pasaria si para darle tiempo a la BD de refrescarse, Implemento un mecanismo que corte la “corriente”(requests) hasta que el sistema se vuelva a estabilizar !!



Y una vez que este estable deje permitir el pase de “corriente”
(requests) nuevamente

Entonces

Circuit Breaker es el componente que me permite abrir o cerrar el circuito cuando se supera un threshold(limite) X. Y ese cierre de circuito se da por un tiempo suficiente para que el sistema se recupere de nuevo

Ejemplo de implementacion:

<https://github.com/balbuenac/circuit-breaker>

Actividad #2: Reliability (30 min)

- Donde podria ayudar un circuit breaker en el sistema de leasing ?
- Que metricas podria usar en mi sistema de colegio para detectar problemas de reliability ?



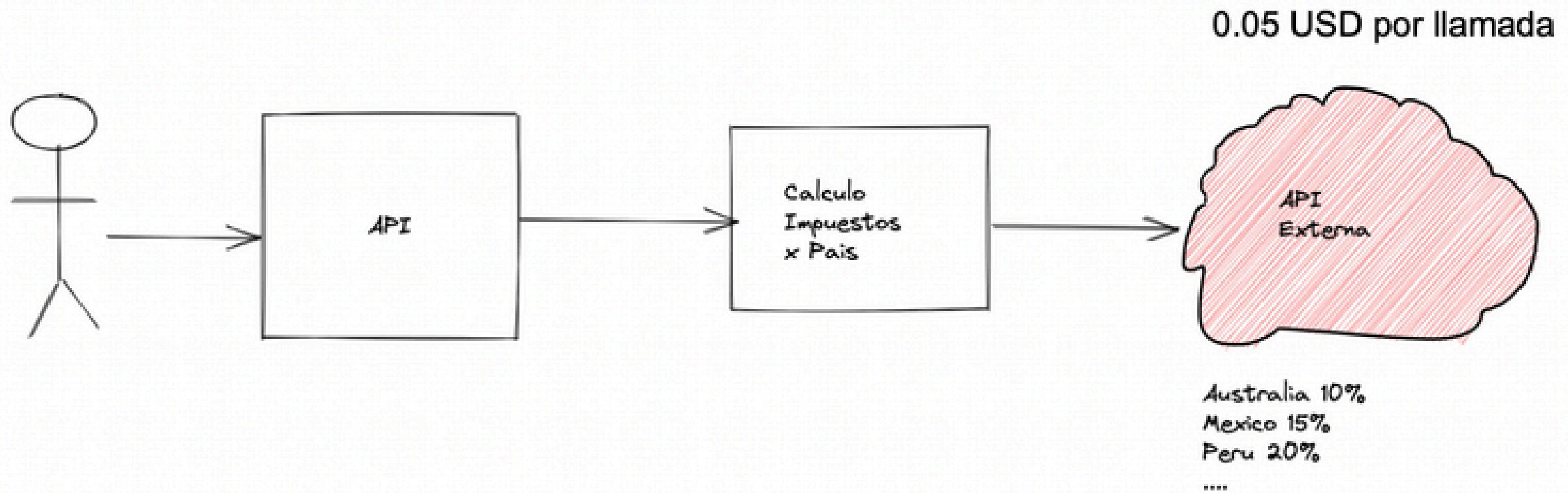
UTEC

CACHING

ARQUITECTURA DE SOFTWARE

Carlos Balbuena

Escenario 1

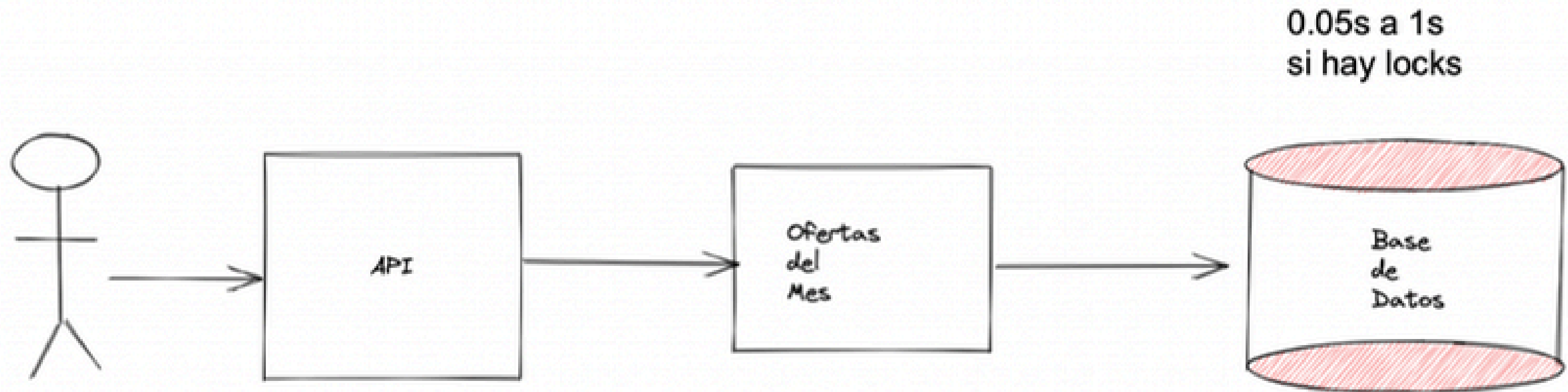


Escenario 1

Llamada API	<u>Calculo</u>	Llamada Api Externa	Costo(USD)
1	1	1	0.05
10	10	10	0.5
100	100	100	5
1000000	1000000	1000000	500000

500,000 dolares ??

Escenario 2



Escenario 2

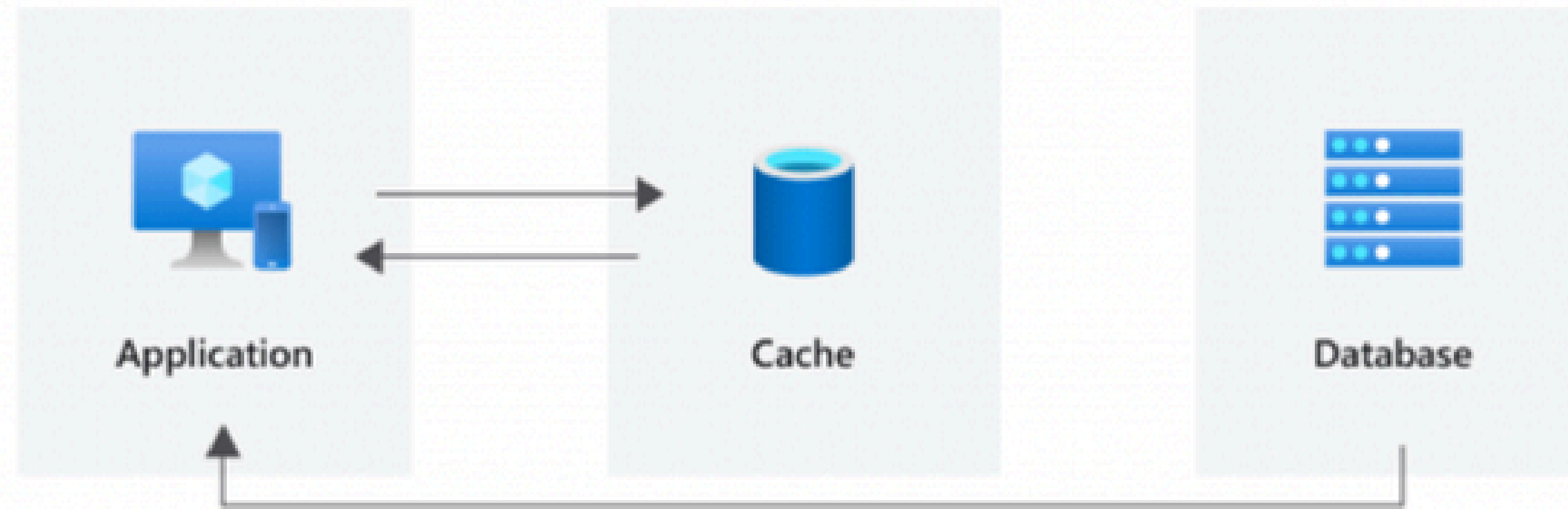
Llamada API	<u>Calculo</u>	Llamada Api Externa	Costo(segundos)
1	1	1	0.05
10	10	10	0.5
100	100	100	5
1000000	1000000	1000000	problemas con locks ?

**La experiencia de usuario
pierde calidad**

Caching es la técnica que permite **salvar en forma temporal** data para su uso inmediato.

Se implementa usando key y values de modo que cuando se requiera obtener la data en el próximo request. Simplemente se obtiene directamente usando la llave.

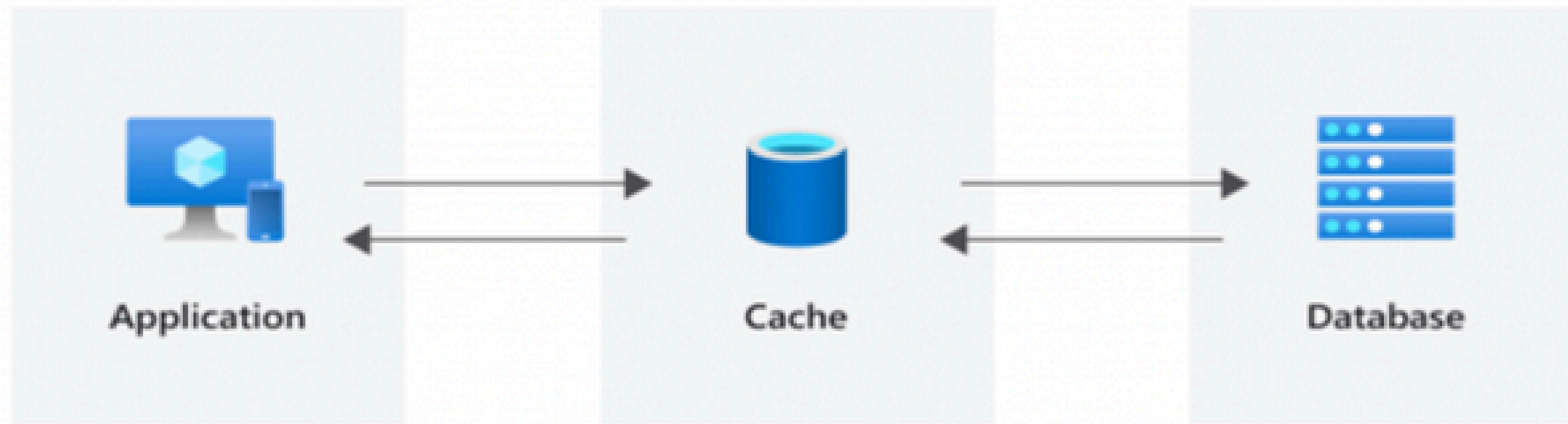
Tipos de Caching



Cache-aside

La aplicacion consulta al Cache pero si no encuentra el valor llama a la API o base de datos para luego guardarlo en cache.

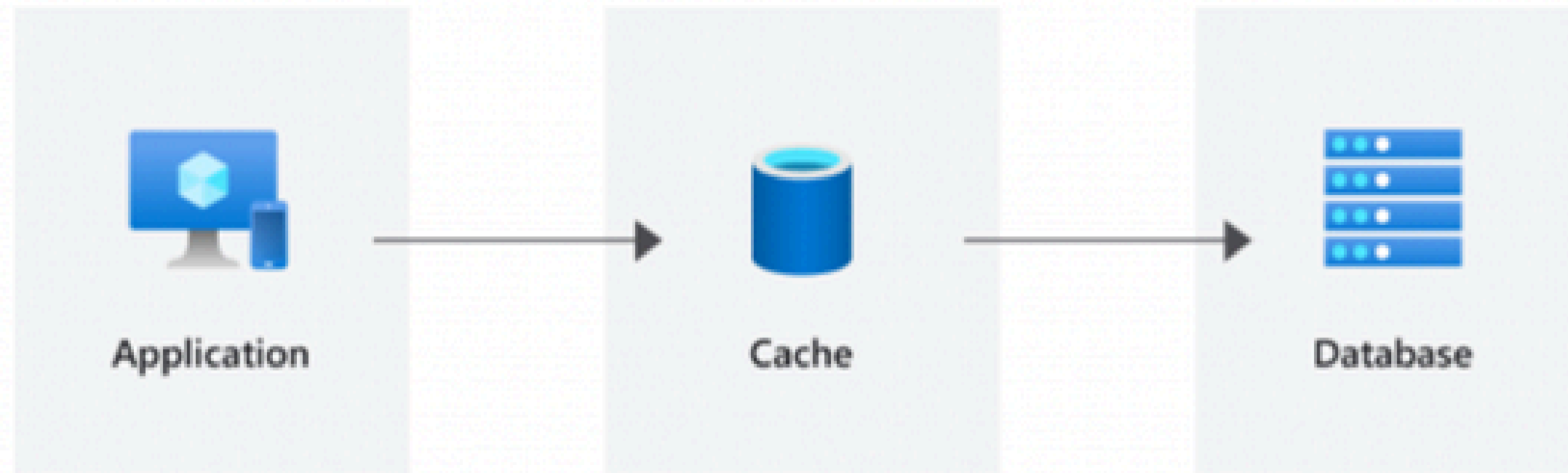
Tipos de Caching



Read-through/write-through cache

La aplicacion consulta al Cache. Y el cache se encarga por sí mismo de actualizar en la base de datos.

Tipos de Caching



Write-behind/write-back cache

La aplicacion escribe a Cache y este se actualiza en la base de datos.

Beneficios

Benefits of caching



Improved application performance

Reading data from an in-memory cache is much faster than accessing data from a disk-driven data store. And with faster access to data, the overall application experience significantly improves.



Reduced database usage and costs

Caching leads to fewer database queries, improving performance and reducing costs by limiting the need to scale database infrastructure and decreasing throughput charges.



Scalable and predictable performance

A single cache instance can handle millions of requests per second, offering a level of throughput and scalability that databases can't match. Caching also offers the flexibility you need whether you're scaling out or scaling up your applications and data stores. Then your application can let many users access the same files simultaneously, without increasing the load on back-end databases. And if an application often experiences spikes in usage and high throughput, in-memory caches can mitigate latency.

En resumen: Mejorar performance

Use Cases

Output caching

Output caching helps increase webpage performance by storing the full source code of pages such as HTML and client scripts that a server sends to browsers for rendering. Every time a user views the page, the server caches the output code in the application's memory. This enables the application to serve requests without running page code or communicating with other servers.

Data caching and database caching

Database speed and throughput can be key factors in overall application performance. Database caching is used for frequent calls to data that doesn't change often, such as pricing or inventory data. It helps websites and applications load faster while increasing throughput and lowering data retrieval latency from back-end databases.

Storing user session data

Application users often generate data that must be stored for short periods. An in-memory data store like Redis is perfect for efficiently and reliably storing high volumes of session data like user input, shopping cart entries, or personalization preferences at a lower cost than storage or databases.

Message brokers and publish/subscribe architectures

Cloud applications often need to exchange data between services, and they can use caching to implement publish/subscribe or message broker architectures that reduce latency and accelerate data management.

Applications and APIs

Like browsers, applications save important files and data to quickly reload that information when needed. Cached API responses eliminate the demand or load on application servers and databases, delivering faster response times and better performance.

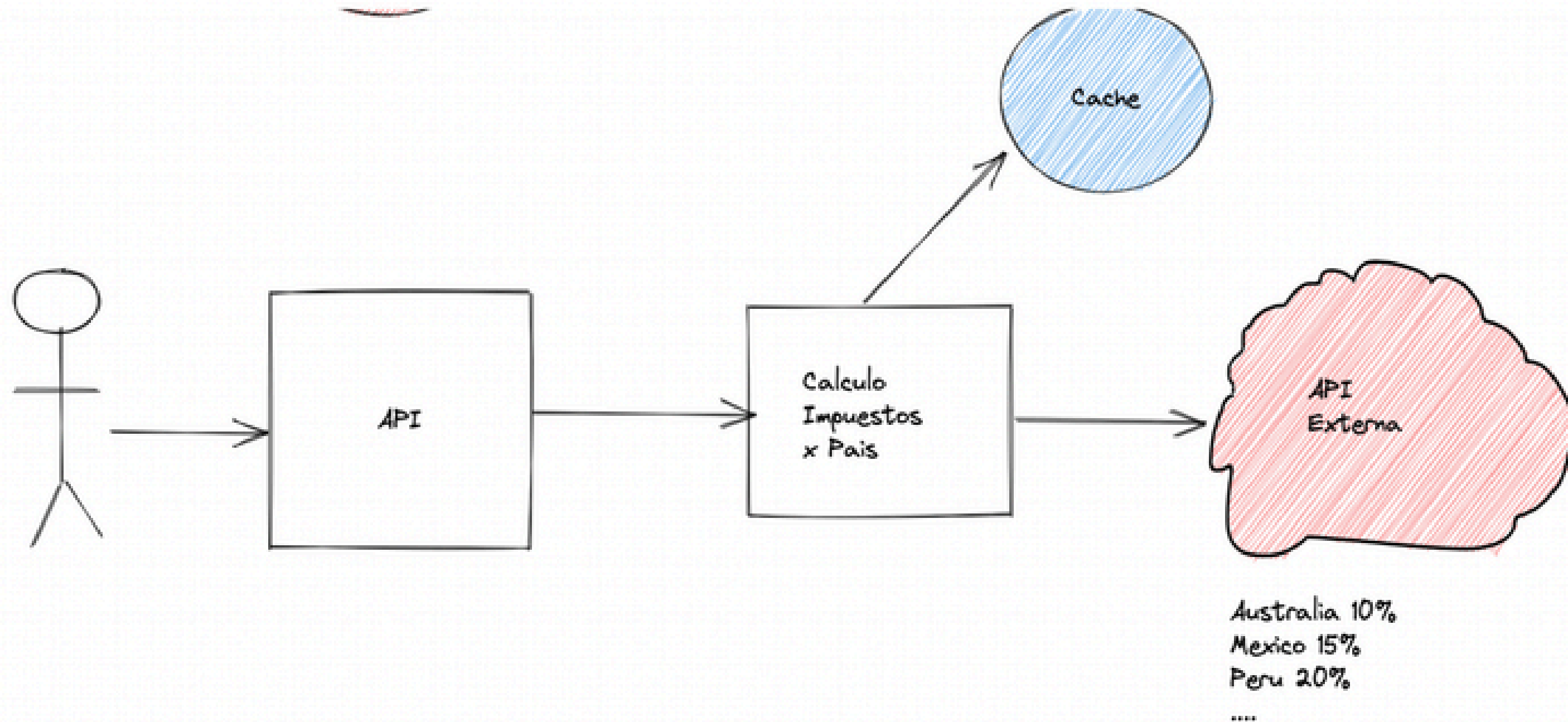
Know your tech

TTL - Time to Live - El tiempo que un “valor” estará en cache.

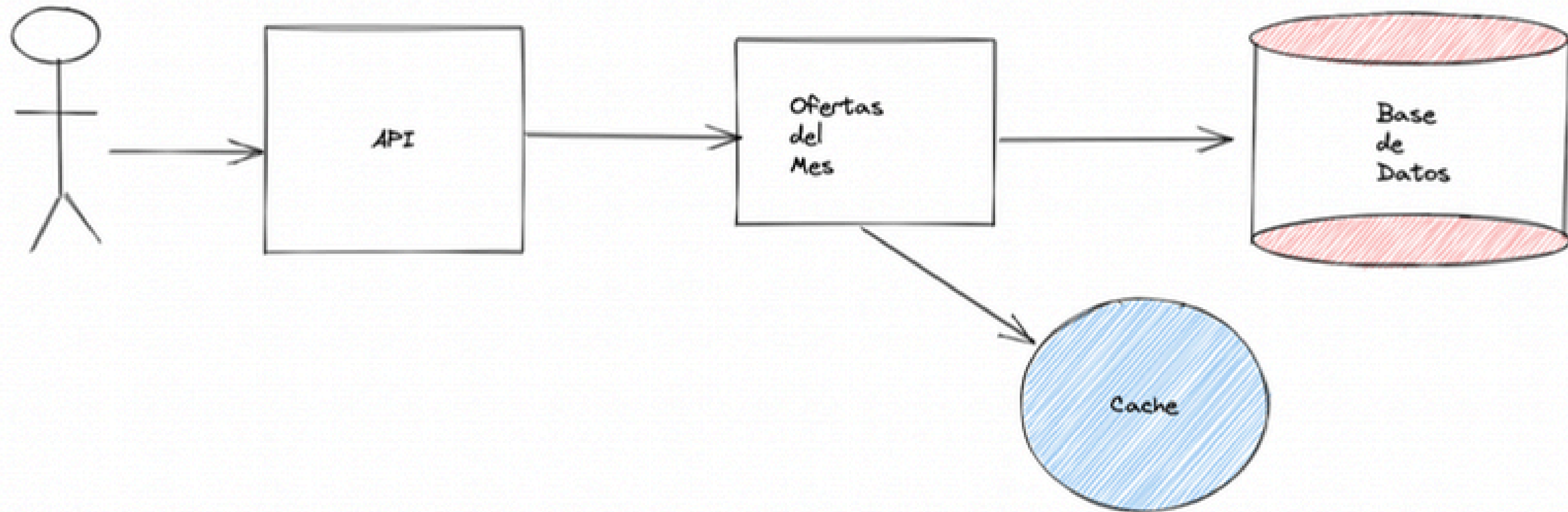
Esto se decide en base a cuan necesario la data que se guardara en cache cambia.

CDN - Content Delivery Network es un tipo de cache de contenido estatico a nivel “regional”.

Mejorar Escenario 1



Mejorar Escenario 2



Cache Implementaciones



Actividad #2: Caching (30 min)

- Donde podria ayudar caching en el sistema de colegio ?
- Que metricas podria usar en mi sistema en general para decidir si usar o no caching ?

Siguiente clase teorica:

- Mas mecanismos de Reliability
- Cache in-memory y distribuida

Pipeline LABs

v2.0

